101 | 参考答案与结果解读

1. 单请求结果

预填充0.003秒，首token时刻2.00395，完成时刻2.00585；TTFT为0.00395，总延迟0.00585，排队等待0。若第二个相同请求在2.001到达，它从2.00585开始，queue_wait为0.00485秒。它的总延迟还要再加自己的预填充和三个解码步骤。

2. 为什么连续批不等于永远更快

静态批不补已经空出的座位，长回答可能让短回答释放的容量浪费。连续批每轮补位，因此可能提高利用率；但本模拟中接纳新请求的预填充会暂停整个活跃批的解码，长提示可能拉高已有请求的等待。哪种策略更好要看负载、容量和目标分位数，不应只根据名字判断。

本模拟没有固定等待凑批窗口，静态批只收当前已经到达的请求；低负载时两者可能接近。所有策略共享相同工作量，差别来自调度规则，而非少生成了一些token。

3. 统计口径

48=4种到达率×3个种子×4种策略；8640=48×180。p95由180条延迟按NumPy默认线性插值计算。1秒超时比例是latency>1的比例。排空吞吐为180除以最后完成减首个到达；若只使用到达窗口，可能把结束后仍在排队的服务时间遗漏，从而夸大吞吐。

有限负载能最终排空，不证明持续相同到达率下队列稳定。尤其最后一批到达以后队列下降，只能说明系统在处理剩余工作。要推断稳态，需要更长负载、稳定性分析和真实服务测量。

4. 训练通信边界

p=1时ring通信两项都为0，步时间为0.08秒，加速与效率均为1。带宽减半只让字节传输项翻倍，不改变启动项或理想计算项；延迟增大则影响$2(p-1)$个阶段的启动成本。不能把两者混为一个总“网络速度”。

公式假设不重叠计算与通信、均匀计算分摊、固定梯度字节数与有效带宽。真实系统可能重叠通信，也可能拓扑不均、拥塞或慢节点，所以模型可高估也可低估步时间。

5. 模型遗漏项与合格结论

遗漏K/V缓存容量、真实注意力复杂度、网络入口、分块预填充、请求取消、失败重试、GPU内核与异构硬件等。不能以模拟器本身执行很快证明服务吞吐很高。合格结论写成“在给定服务时间和负载假设下，策略产生了这些排队与尾延迟差异；需要真实阶段测量校准后再用于容量规划”。

模拟结果的精确值见summary.csv与requests.csv。由于这些数值是确定性事件推进，独立重放应逐位或浮点容差内一致；只有simulator_cpu_wall_seconds会随机器负载变化。

6. 本次容量模拟的具体结果

以下为假定到达率60请求/秒时，三个负载种子的统计量算术均值。p95列是“三个p95的均值”，不是把540请求混合后的p95。所有服务秒数与吞吐都是模拟结果，不是硬件实测。

策略	模拟p95延迟均值（秒）	模拟排空吞吐均值（请求/秒）	模拟超过1秒比例均值
serial	4.7260	22.544	80.56%
static4	2.5424	31.599	67.04%
continuous4	1.8223	36.851	54.44%
continuous8	1.4317	41.156	44.44%

增大批容量并不消除所有等待：串行预填充仍会暂停活跃解码，有限负载还需要排空。请把这张表与queue_trace.csv同时读，区分高负载阶段的积压与到达结束后的清空。策略相对优劣只在上述服务时间常数、长度分布与调度规则下成立；改变长提示比例或预填充分块规则应另做实验。

一个具体差异是：continuous8相对continuous4的模拟排空吞吐从约36.85增至41.16请求/秒，远没有随容量翻倍；同时仍有约44.44%的请求超过假定1秒阈值。因此本负载下“加座位”改善了利用率，却没有使服务达到阈值。这里应保留未达标结论，而不是只突出吞吐提升。